4팀

원격 근무용 업무처리 프로그램

지능형 원격 근무 관리 및 협업 플랫폼
MFC 기반 업무 시스템과 라즈베리파이를 연동한
실시간 근무 상태 모니터링 시스템

팀장: 강승진
팀원: 김주엽, 김찬송, 송현수, 이문형

목차

- 프로젝트 개요
- 프로젝트 팀 구성 및 역할
- 프로젝트 수행 절차 및 방법
- 프로젝트 수행 경과

선정 배경

저희는 원격 근무 환경의 "협업 단절"과 "관리의 공백"이라는 두 가지 핵심 문제를
동시에 해결할 수 있는 통합적인 시스템이 필요하여 이 프로젝트를 선정하게 되었음.

기획 의도

협업 효율 극대화를 위한 "통합 업무 플랫폼 구축"과 투명한 근무 관리를 위한 "지능형 모니터링 시스템"구축

기대 효과

- 생산성 향상 및 데이터 기반 경영
- 업무 몰입도 증가
- 신뢰 기반의 자율성 확보

활용 방안

- 원격/하이브리드 근무 기업 도입
- BPO 및 고객센터 운영
- 온라인 교육 및 시험 감독
- SaaS (서비스형 소프트웨어) 모델

기술 스택



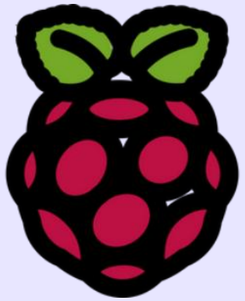








Visual Studio Code







팀 구성 및 역할



01

팀장: 강승진

역할 : **DB**구축, 라즈베리파이 서보 센서, 전체 통신



02

팀원: 김주엽

역할 : **flask,**라즈베리파이 **LED** 제어



03

팀원: 김찬송

역할 : **MFC** 커뮤니티, 메신저



04

팀원: 송현수

역할 : **OpenCV** 얼굴 인식 및 학습



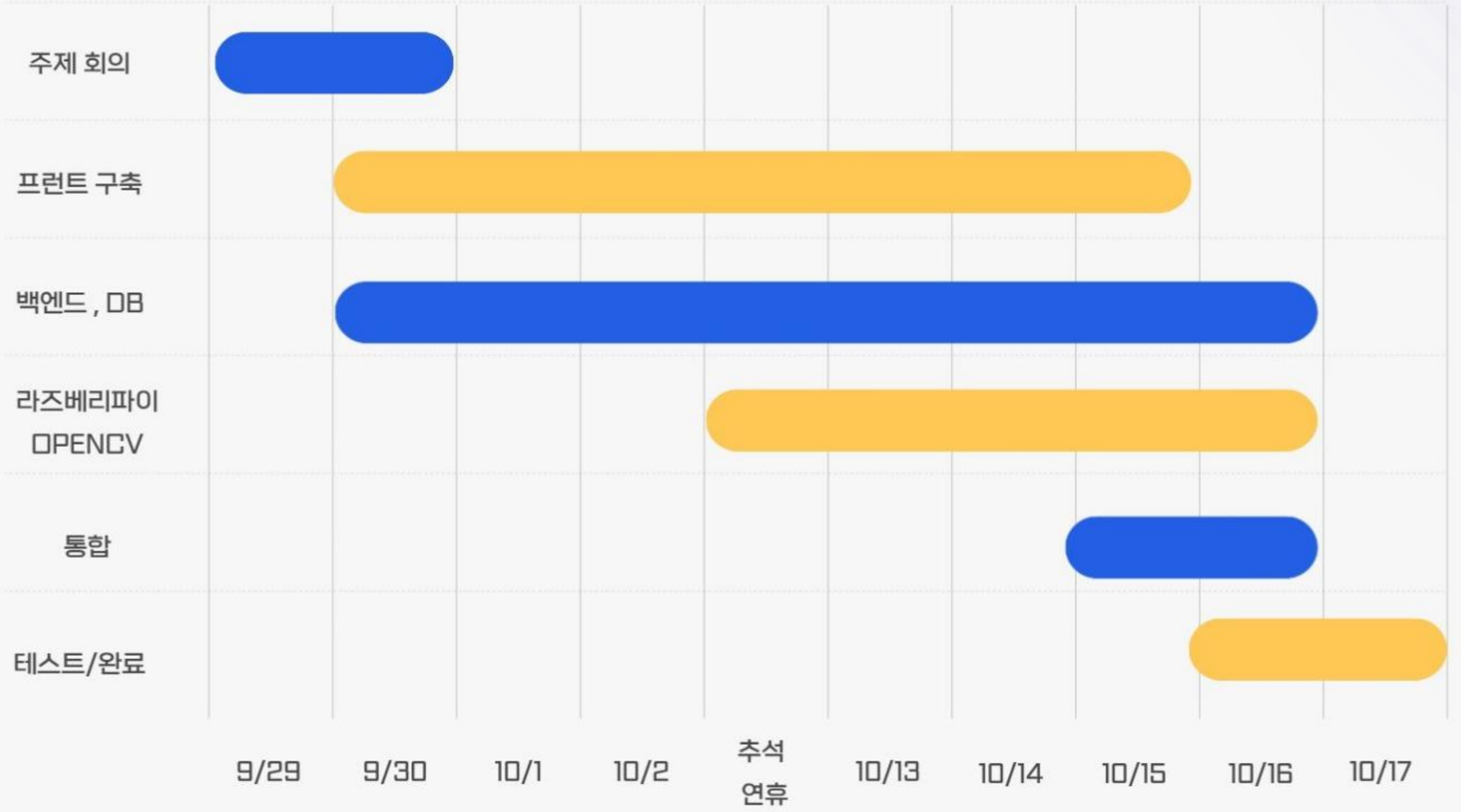
05

팀원: 이문형

역할 : **MFC** 업무처리, 사원

간트 차트

원격근무용 업무 처리 시스템



ERD - 다이어그램

총12개 테이블

사원 테이블

사용자 테이블

채팅방 테이블

참여자 테이블

메시지 테이블

공지사항 테이블

익명 게시판 테이블

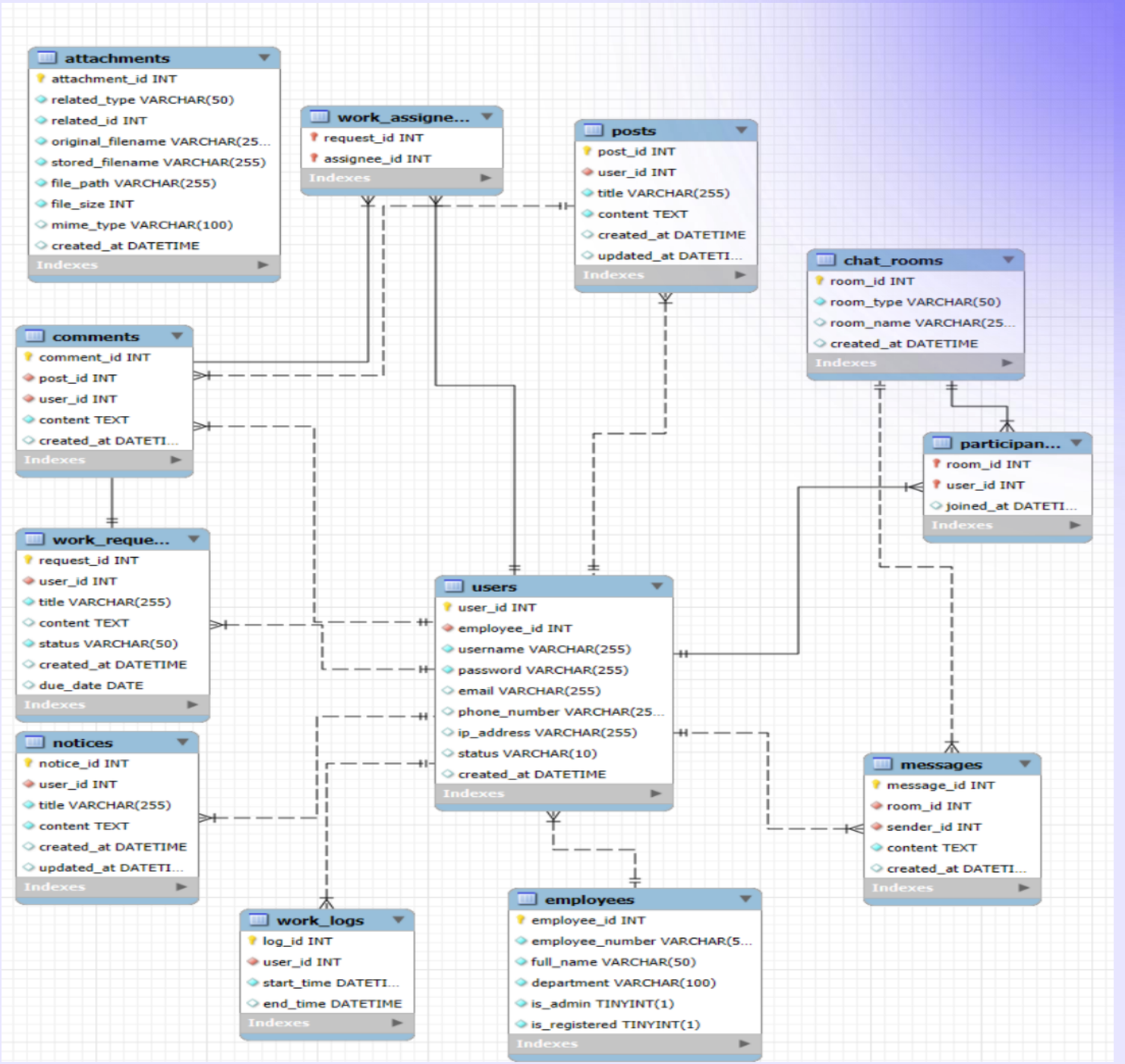
댓글 테이블

첨부파일 테이블

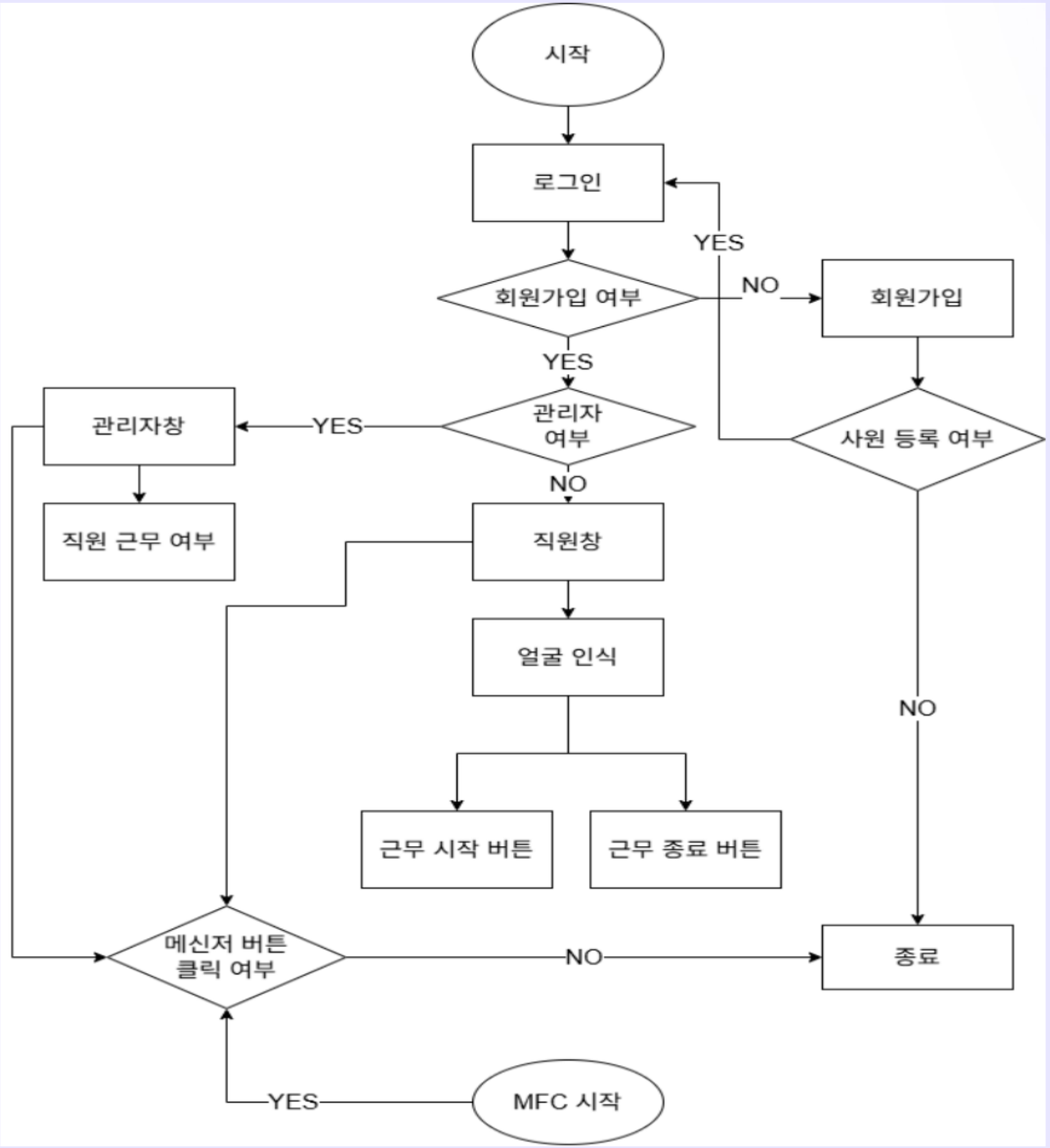
업무 요청 테이블

업무 담당자 테이블

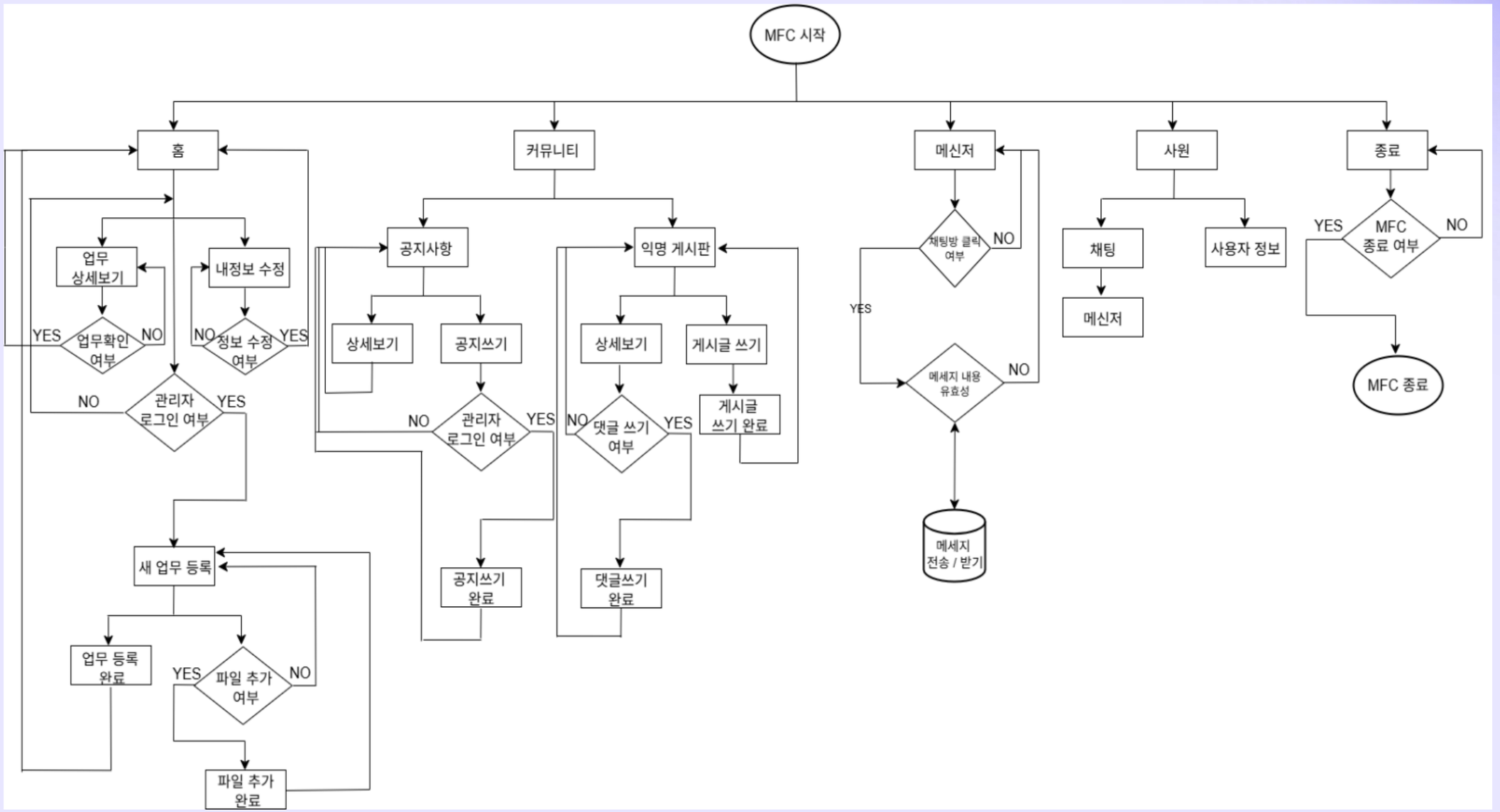
출퇴근 기록 테이블



HTML 순서도


시작
로그인
YES
회원가입 여부
NO
회원가입
YES
관리자창
YES
관리자
여부
사원 등록 여부
NO
직원 근무 여부
직원창
얼굴 인식
NO
근무 시작 버튼
근무 종료 버튼
메신저 버튼
클릭 여부
NO
종료
YES
MFC 시작

MFC 순서도

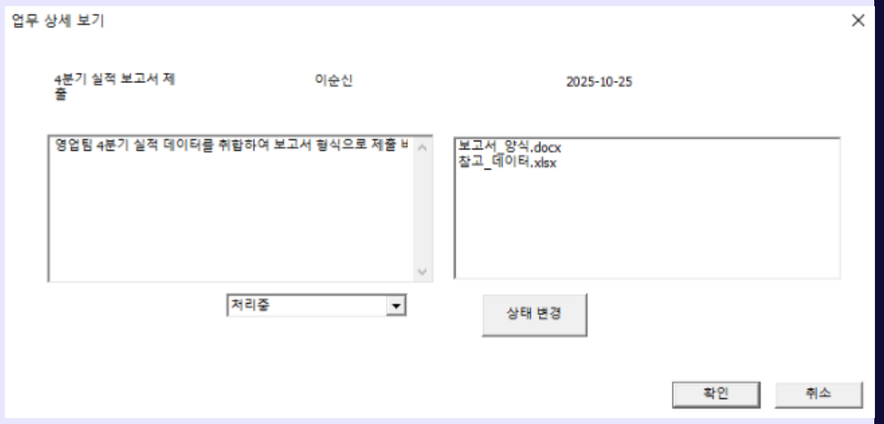

MFC 시작
홈
업무 상세보기
YES
업무확인 여부
NO
내정보 수정
NO
정보 수정 여부
YES
NO
관리자 로그인 여부
YES
새 업무 등록
업무 등록 완료
YES
파일 추가 여부
NO
파일 추가 완료
커뮤니티
공지사항
상세보기
공지쓰기
NO
관리자 로그인 여부
YES
공지쓰기 완료
익명 게시판
상세보기
게시글 쓰기
NO
댓글 쓰기 여부
YES
게시글 쓰기 완료
댓글쓰기 완료
메신저
채팅방 클릭 여부
NO
YES
메세지 내용 유효성
NO
메세지 전송 / 받기
사원
채팅
메신저
사용자 정보
종료
YES
MFC 종료 여부
NO
MFC 종료

문형 파트









로그인한 사용자의 권한(관리자/일반 직원)에 맞춰 개인화된 업무목록 및 사용자의 정보확인가능.

버튼을 이용하여 업무 상세보기 창, 새 업무 등록 창, 내 정보 수정 창으로 이동가능.

내 정보 수정 – 전화번호, 이메일, 비밀번호 변경가능
새 업무 등록(관리자 전용) – 관리자만 보이는 버튼으로 새로운 업무를 작성가능
업무 상세 보기 – 선택한 업무 하나에 대한 모든 상세 정보를 조회가능

HomeDlg 주요함수

```
void CHomeDlg::LoadCurrentUserInfo() { ... }
```

```
void CHomeDlg::OnLButtonDown(UINT nFlags, CPoint point) { ... }
```

WorkDetailDlg 주요함수

```
void CWorkDetailDlg::LoadWorkDetails() { ... }
```

```
void CWorkDetailDlg::OnBnClickedButtonUpdateStatus() { ... }
```

NewWorkRequestDlg 주요함수

```
void CNewWorkRequestDlg::LoadEmployeeList() { ... }
```

```
void CNewWorkRequestDlg::OnBnClickedOk() { ... }
```

MyInfoDlg 주요함수

```
void CMyInfoDlg::LoadCurrentUserData() { ... }
```

```
void CMyInfoDlg::OnBnClickedChangePhone() { ... }
```

LoadCurrentUserInfo(): DB에서 현재 로그인한 사용자의 정보를 가져와 멤버 변수에 저장

OnLButtonDown(...): 클릭된 좌표를 확인하여 해당하는 새 창을 **DoModal()** 함수로 띄움.

LoadWorkDetails(): DB에서 해당 업무의 모든 상세 정보(제목, 내용, 첨부파일 목록 등)를 화면에 출력.

OnBnClickedButtonUpdateStatus(): 관리자가 업무 상태를 변경 가능.

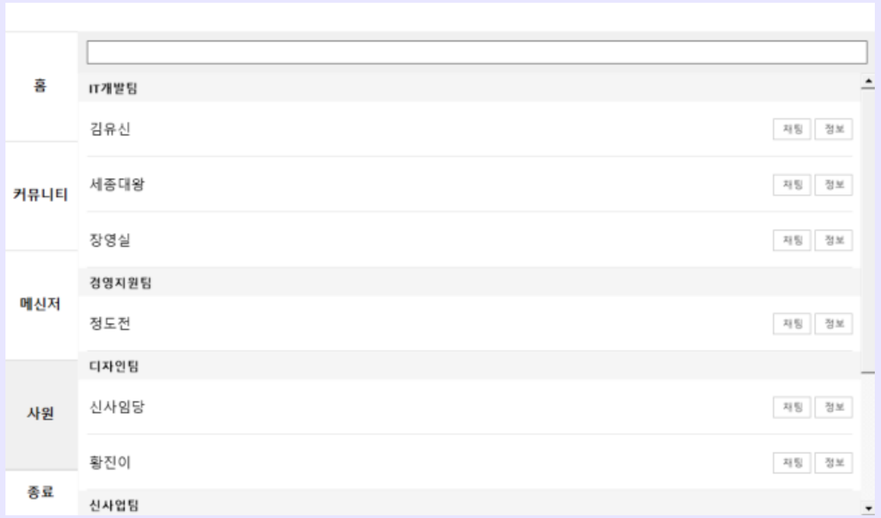
LoadEmployeeList(): 창이 열릴 때, 업무를 할당할 수 있는 직원 목록출력.

OnBnClickedOk(): '등록' 버튼을 누르면 각각의 정보를 확인이후 **DB**에 저장.

LoadCurrentUserData(): 창이 열릴 때, **DB**에서 현재 로그인한 사용자의 이메일과 전화번호를 출력.

OnBnClickedChangePhone(): '변경' 버튼을 누르면, 해당 입력 상자의 텍스트를 가져와 **DB**에 있는 사용자 정보를 수정.

FriendsListDlg



FriendsListDlg 주요함수

```
void CFriendsListDlg::OnEnChangeSearch( ) { ... }
```

```
int CFriendsListDlg::GetOrCreateDirectChatRoom( int targetUserId) { ... }
```

```
void CFriendsListDlg::OnLButtonDown(UINT nFlags, CPoint point) { ... }
```

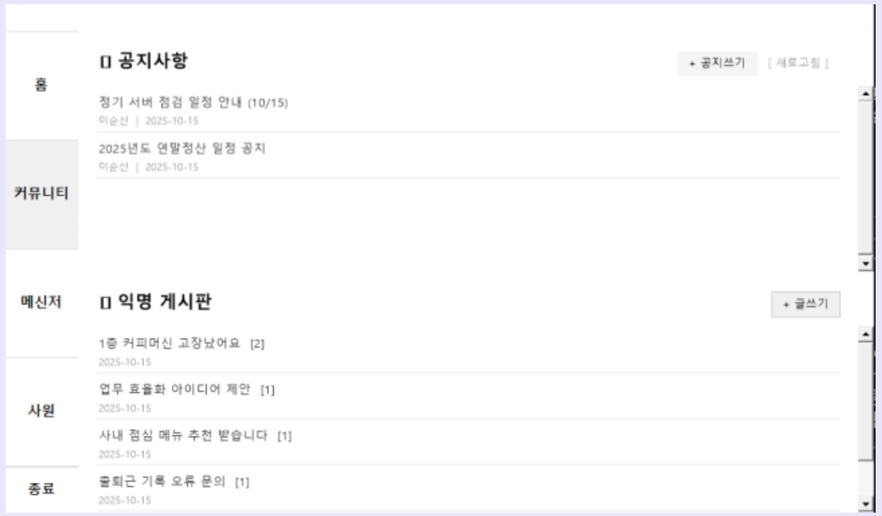
전 직원의 목록을 부서별로 보여주고, 이름으로 검색하며, 특정인의 상세 정보를 보거나 **1:1** 채팅가능.

주요함수

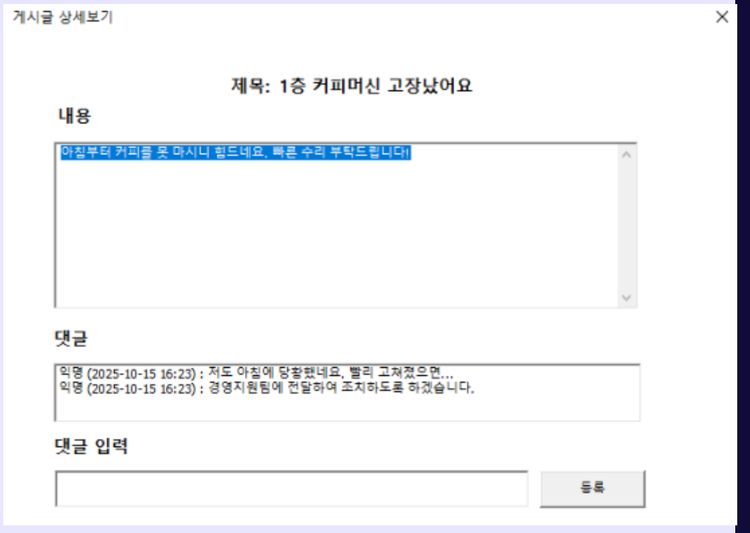
OnEnChangeSearch(): 검색창에 글자가 입력될 때마다 호출하여 입력된 글자가 포함된 직원만 화면에 다시 그림

OnLButtonDown() & GetOrCreateDirectChatRoom(): DB에 접속하여 두 사용자(나, 상대방)만 참여하는 **1:1** 채팅방이 이미 있는지 확인 이후 채팅방이 없다면 테이블에 새 방을 만들고 채팅화면으로 이동

CommunityDlg







찬송 파트

화면을 위아래로 나누어, 상단에는 '공지사항'을, 하단에는 '익명 게시판'을 표시하는 두 가지 기능을 동시에 제공.
또한 버튼을 이용하여 공지사항 및 익명 게시판확인 및 작성으로 이동가능

공지쓰기 (관리자 전용): 관리자에게만 보이는 버튼으로, 클릭 시 회사 전체에 알릴 새로운 공지사항을 작성가능.

글쓰기: 익명 게시판 하단에 있는 버튼으로, 누구나 익명으로 새로운 게시글을 작성가능.

공지사항 상세확인: 선택한 공지사항 하나에 대한 모든 상세 정보를 조회가능

익명게시판 상세확인: 선택한 익명게시판 하나에 대한 모든 상세 정보를 조회가능

CommunityDlg 주요함수

```
void CCommunityDlg::LoadDataFromDB( ) { ... }
```

```
void CCommunityDlg::OnPaint( ) { ... }
```

```
void CCommunityDlg::OnLButtonDown(UINT nFlags, CPoint point) { ... }
```

NoticeDetailDlg 주요함수

```
BOOL CNoticeDetailDlg::OnInitDialog( ) { ... }
```

WriteNoticeDlg 주요함수

```
void CWriteNoticeDlg::OnBnClickedOk( ) { ... }
```

PostDetailDlg 주요함수

```
void CPostDetailDlg::LoadPostDetails( ) { ... }
```

WritePostDlg 주요함수

```
void CWritePostDlg::OnBnClickedOk( ) { ... }
```

CommunityDlg::LoadDataFromDB(): **DB**에서 '공지사항', '익명 게시판' 목록을 가져와 화면에 출력

CommunityDlg::OnPaint(): 상단과 하단 두 개의 독립된 그리기 영역을 만들고 관리자일 경우에만 상단 헤더 영역에 공지쓰기 버튼 출력.

CommunityDlg::OnLButtonDown(): 서 저장해 둔 새로고침, 공지쓰기, 글쓰기 버튼의 좌표와 클릭 좌표를 비교하여 해당일치하는 버튼이 있으면 해당 기능을 실행.

NoticeDetailDlg::OnInitDialog(): **DB**에서 공지 데이터를 가져와 화면에 표시.

WriteNoticeDlg::OnBnClickedOk():'등록' 버튼을 누르면 호출되어, 사용자가 입력한 제목과 내용을 **DB**의 **notices** 테이블에 **INSERT** 하는 역할.

PostDetailDlg::LoadPostDetails(): 창이 열리거나 새 댓글이 달렸을 때 호출되어, 게시글 본문과 모든 댓글을 **DB**에서 가져와 화면을 구성.

WritePostDlg::OnBnClickedOk(): '등록' 버튼을 누르면 호출되어, 사용자가 입력한 제목과 내용을 **DB**에 삽입하는 역할.

MessengerDlg



MessengerDlg 주요함수

```
void CMessengerDlg::LoadChatRoomsFromDB( ) { ... }
```

```
void CMessengerDlg::LoadMessagesFromDB(int roomId) { ... }
```

```
void CMessengerDlg::OnBnClickedSend( ) { ... }
```

```
void CMessengerDlg::OnTimer(UINT_PTR nIDEvent) { ... }
```

다른 동료와 **1:1** 대화를 하거나 여러 명과 그룹 대화를 할 수 있습니다. 다른 사람이 보낸 메시지를 주기적으로 확인하여 자동으로 화면을 갱신하여 실시간 채팅기능 구현.

LoadChatRoomsFromDB(): 현재 사용자가 참여하고 있는 모든 채팅방의 목록을 가져오며 가장 마지막에 온 메시지 하나를 추가로 조회하여 미리보기 텍스트로 사용.

LoadMessagesFromDB():
사용자가 왼쪽 목록에서 특정 채팅방을 클릭하면, 해당 채팅방의 **ID**를 전달받아 모든 메시지를 시간순으로 가져옴.

OnBnClickedSend(): 사용자가 '전송' 버튼을 누르면 입력창의 텍스트를 읽어 대화내용에 출력

OnTimer(): 몇 초에 한 번씩 자동으로 호출하여 화면 초기화를 시킴. 사용자가 실시간처럼 메시지를 받는것처럼 표현가능.

승진

```
CROW_ROUTE(app, "/retrain").methods(crow::HTTPMethod::Post)
       ([]() {
           update_registered_embeddings();
           crow::json::wvalue res({{"success", true}, {"message", "얼굴 특징 정보가 성공적으로 갱신되었습니다."}});
           return crow::response(200, res);
       });
```

```
// --- API 2: /detect ---
        CROW_ROUTE(app, "/detect").methods(crow::HTTPMethod::Post)
        ([&](const crow::request& req) {
            crow::multipart::message msg(req);
            auto username_part = msg.get_part_by_name("target_username");
            if (username_part.body.empty()) return crow::response(400, "target_username missing");
            std::string target_name = username_part.body;
```

```
img_bytes = base64.b64decode(data_image.split(',')[1])
payload = {
    'target_username': (None, image_sender_username),
    'image': ('image.jpg', img_bytes, 'image/jpeg')
}
res = requests.post(RPI_DETECT_URL, files=payload, timeout=1)

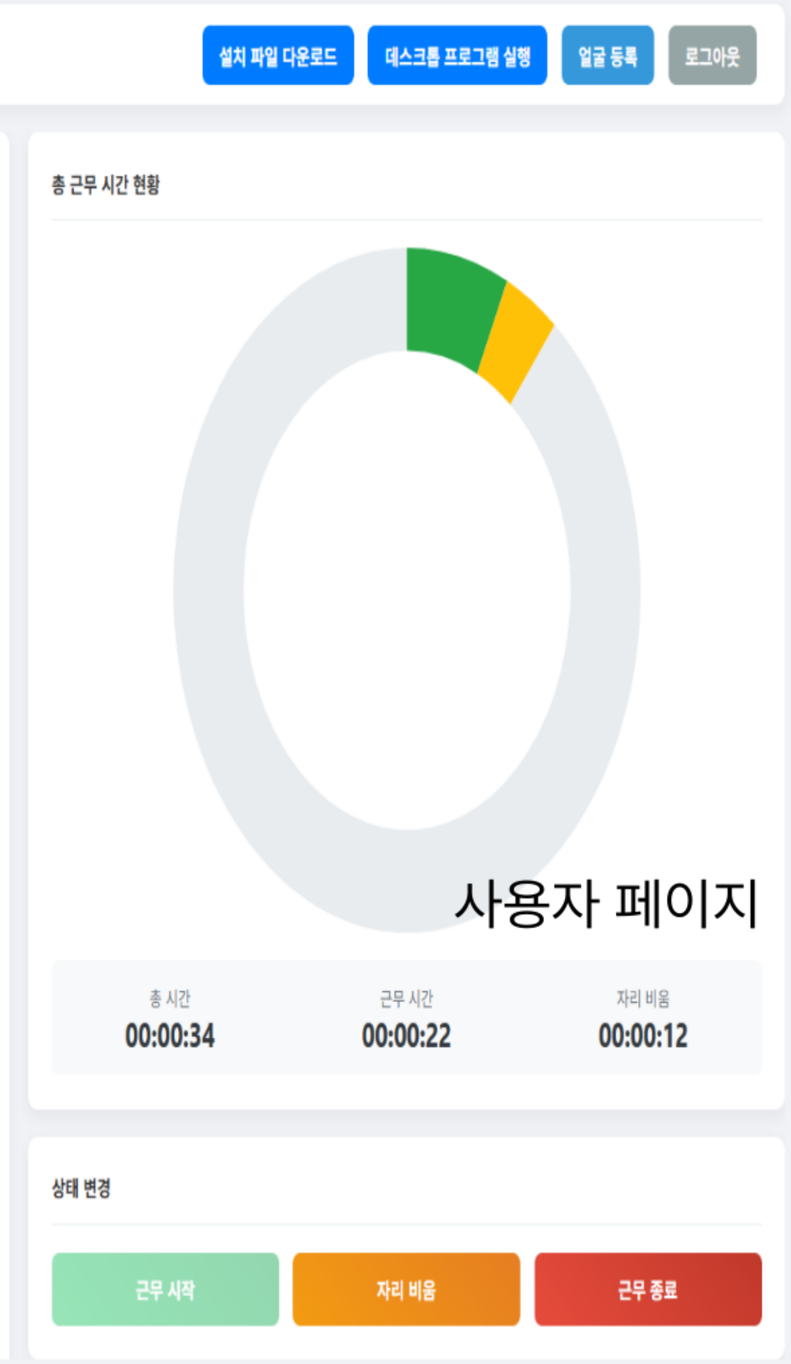
result_image = ''
is_drowsy = False
face_detected = True # 기본값은 True로 설정하여 네트워크 오류 시 이탈로 간주하지 않도록 함
if res.status_code == 200:
    response_data = res.json()
    result_image = response_data.get('image', '')
    is_drowsy = response_data.get('is_drowsy', False)
    face_detected = response_data.get('face_detected', True) # 🌟 C++ 서버로부터 값 수신
```

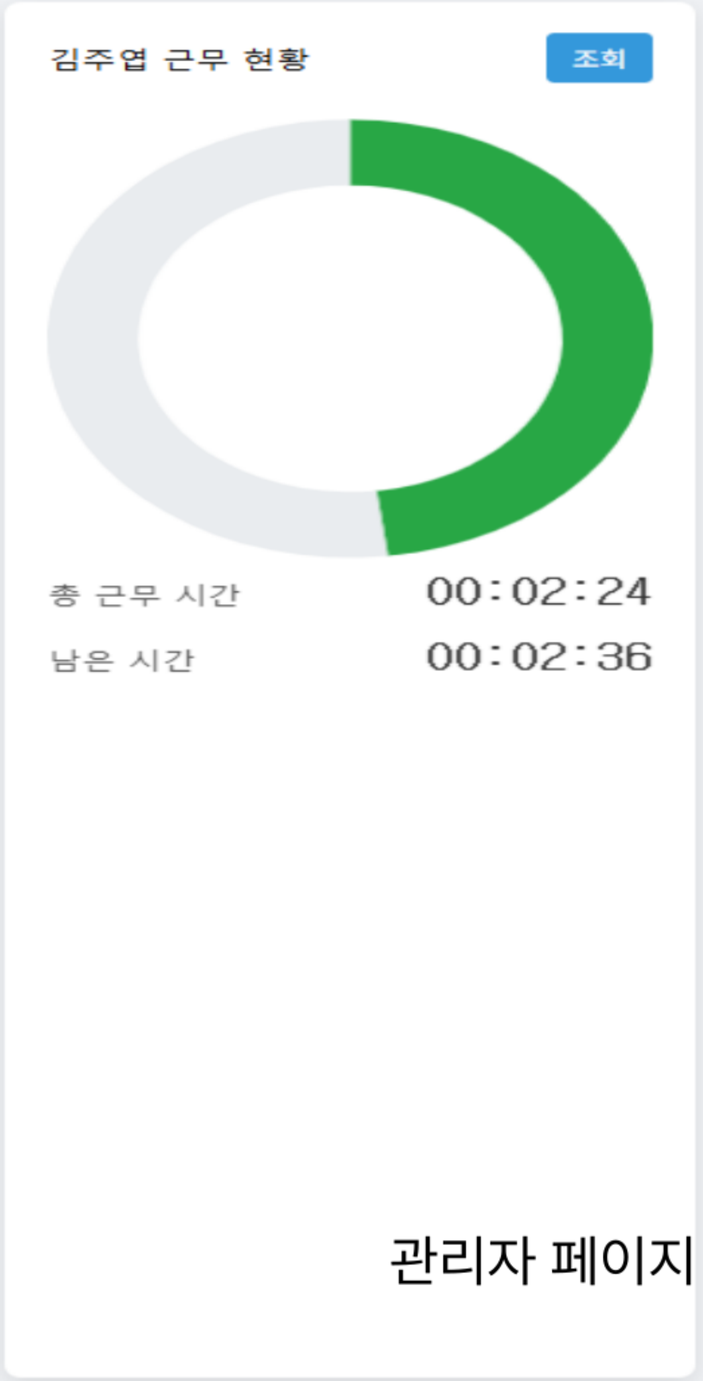
Flask 서버와 **Crow C++** 서버 사이는 **HTTPS**통신 **(REST API)**을 사용
사용자 브라우저와 **Flask** 서버 사이는 웹소켓 **(WebSocket)** 통신을 사용

분석할 대상의 이름**(target_username)**과 이미지 파일을 한데 묶어 **payload**라는 만듬.
requests.post를 통해 이 요청서를 **C++** 서버와 통신

Flask 서버가 보낸 요청을 여기서 통신

C++ 서버가 분석을 마치면, **Flask** 서버는 그 결과를 다시 돌려받는 코드





1. 원격 업무 처리 시스템의 근무 상태 관리

관리자 페이지에서 특정 직원을 클릭

브라우저가 서버의 주소를 통해 요청

서버에서는 요청을 받아 **DB**에 저장된 데이터를 조회

데이터 분석과 실시간 제어를 통해 완성형 관리 시스템을 구축

```python
#라즈베리파이 LED 서버 소켓 통신 함수
def send_command_to_pi(command):
    """
    라즈베리파이의 C++ LED 서버에 TCP 소켓통신을 이용해서 명령어를 전송
    """
    with pi_socket_lock:
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(2.0)
                print(f"Connecting to RPi LED server at {RPI_LED_SERVER_HOST}:{RPI_LED_SERVER_PORT}...")
                s.connect((RPI_LED_SERVER_HOST, RPI_LED_SERVER_PORT))

                print(f"✅ Sending command to RPi: '{command}'")
                s.sendall(command.encode('utf-8'))

                response = s.recv(1024)
                print(f"✅ Received response from RPi: '{response.decode('utf-8')}'")

        except socket.timeout:
            print(f"❌ ERROR: Connection to RPi server timed out.")
        except ConnectionRefusedError:
            print(f"❌ ERROR: Connection refused. Is the C++ server running on the RPi?")
        except Exception as e:
            print(f"❌ ERROR: An unexpected error occurred while sending command to RPi: {e}")
```

2. 플라스크 서버와 라즈베리파이의 **LED**제어 실시간 연결

-근무 시작 로직**-**

근무시간을 정확히 측정하기 위해 **work_log** 테이블에 상태 변경 시간을 기록함

실시간으로 현재 근무 상태를 시각적으로 보여주기 위해 라즈베리파이로 명령어를 전송

-근무 종료 로직**-**

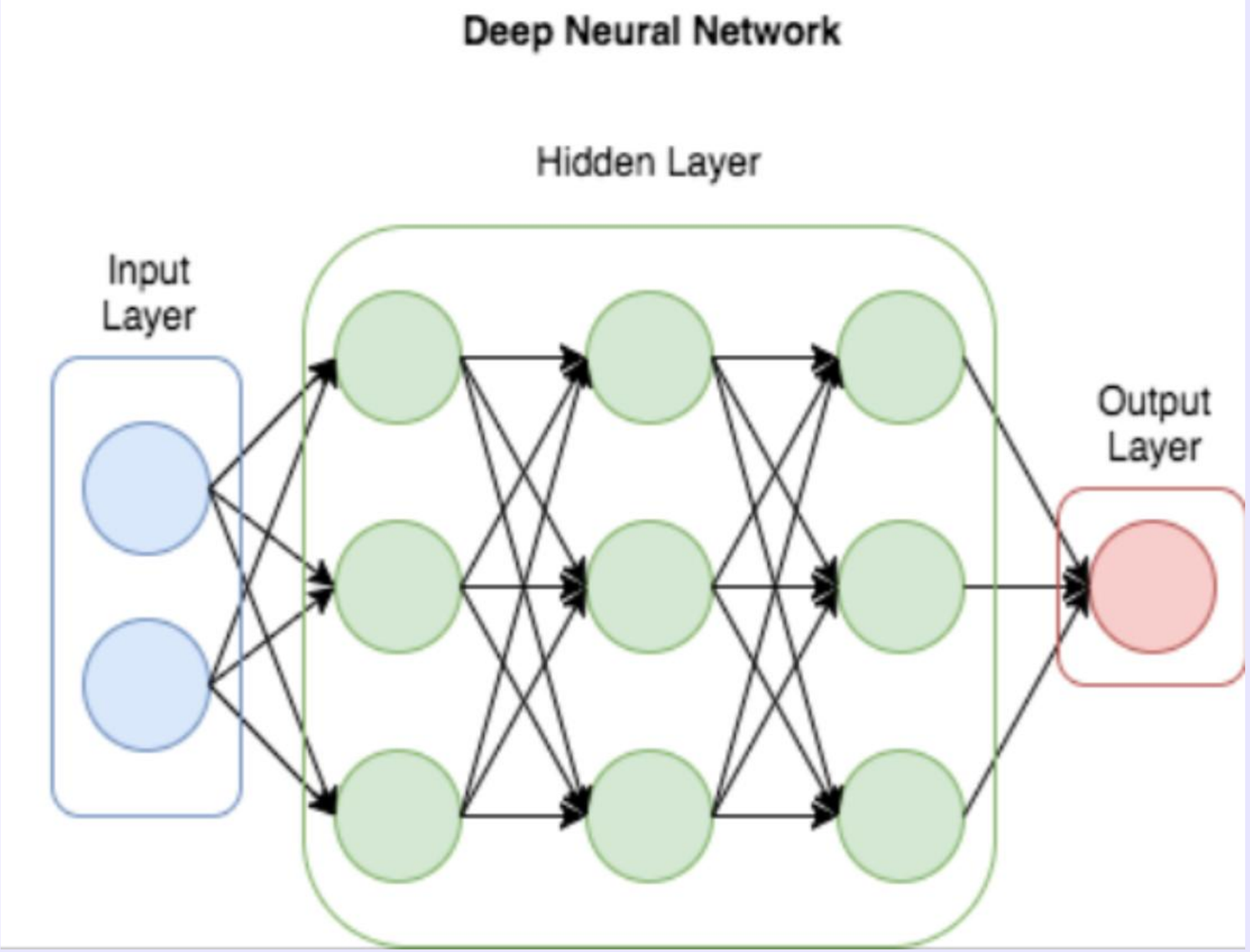
clock_out 함수를 이용해 오늘의 총 근무 시간을 계산

만약 총 근무 시간이 설정한**(5**분**)** 기준에 도달하면 업무 목표 달성을 의미하는 **work_complete**명령을 보냄

-소켓통신을 이용한 **LED**제어 부분**-**

웹서버는 파이썬을 사용하고 라즈베리파이는 **C++**언어를 사용 즉 서로 다른 환경에서 동작하기 때문에 이를 이어줄 표준 **TCP** 소켓 방식을 사용

OPENCV활용 DNN모델 개념



데이터를 여러 방식으로 이리저리 비틀어보고 선을 그어보는 시행착오를 수없이 반복하여, 가장 완벽하게 그룹을 나누는 기준을 스스로 찾아내는 딥러닝 모델

OpenCV 모델 비교

기술명	모델이름	정확도(100번)
DNN	ArcFace	91%
DNN(경량화 CNN)	YuNet모델	97%
DNN	MoblileFaceNet	94%
머신러닝	하르 캐스케이드	76%
Dlib(딥러닝)	68FaceLandmark	87%

현수 파트 코드 부분

```
void load_dnn_models() {
    try {
        // --- [핵심 수정] 파일 이름을 새로 다운로드한 2022년 버전으로 변경 ---
        std::string detector_path = "face_detection_yunet_2022mar.onnx"; [cite: 210]
        std::string recognizer_path = "face_recognition_sface_2021dec.onnx"; [cite: 211]

        // ... 파일 존재 여부 확인

        // OpenCV 4.6.0 호환성을 위한 추가 파라미터는 그대로 유지
        face_detector = cv::FaceDetectorYN::create(detector_path, "", cv::Size(320, 320), 0.9f, 0.3f, 5000); [cite: 213]
        face_recognizer = cv::FaceRecognizerSF::create(recognizer_path, ""); [cite: 213]
        std::cout << "[INFO] 딥러닝 모델 (YuNet, SFACE) 로드 성공." << std::endl; [cite: 214]
    }
    catch (const cv::Exception& e) {
        std::cerr << "[FATAL ERROR] DNN 모델 로드 실패: " << e.what() << std::endl; [cite: 215]
        exit(1); [cite: 216]
    }
}
```

1. 딥러닝 모델 로딩 및 초기화 **(**페이스 서버**1)**
첫 번째 핵심 코드는 얼굴 탐지 및 인식을 위한 딥러닝 모델 파일**(.onnx)**을 로드하고, 프로그램이 사용할 수 있도록 초기화하는 부분입니다**.** 이 과정은 전체 시스템의 가장 기본이 되는 준비 단계입니다**.**

코드 설명

face_detection_yunet_2022mar.onnx는 얼굴의 위치를 찾는 탐지 모델**(YuNet)**입니다**.**
face_recognition_sface_2021dec.onnx는 탐지된 얼굴이 누구의 것인지 식별하기 위한 특징**(**임베딩**)**을 추출하는 인식 모델**(SFACE)**입니다**.**
cv::FaceDetectorYN::create와 **cv::FaceRecognizerSF::create** 함수를 통해 이 모델들을 메모리에 올립니다**.** 이 코드가 성공적으로 실행되어야 서버의 모든 얼굴 인식 기능이 정상적으로 동작할 수 있습니다

현수 파트 코드 부분

```
void update_registered_embeddings() {
    std::lock_guard<std::mutex> lock(model_mutex);
    registered_user_embeddings.clear();
    std::string data_path = "/home/moble03/face";
    // ... 폴더 존재 여부 확인 ...

    for (const auto& user_entry : fs::directory_iterator(data_path)) {
        // ... 사용자별 폴더 순회 ...
        std::string user_name = user_entry.path().filename().string(); [cite: 219, 265]
        std::vector<cv::Mat> user_embeddings;

        for (const auto& image_entry : fs::directory_iterator(user_entry.path())) { [cite: 220, 266]
            // ... 이미지 파일 처리 ...
            cv::Mat aligned_face;
            face_recognizer->alignCrop(image, faces.row(0), aligned_face); [cite: 222, 268]

            cv::Mat feature;
            face_recognizer->feature(aligned_face, feature);
            user_embeddings.push_back(feature.clone());
        }

        if (!user_embeddings.empty()) {
            cv::Mat avg_embedding = user_embeddings[0]; [cite: 223]
            // ... 여러 임베딩의 평균 계산 ...
            cv::normalize(avg_embedding, avg_embedding); [cite: 225, 270]
            registered_user_embeddings[user_name] = avg_embedding;
        }
    }
}
```

2. 등록된 사용자 얼굴 정보 업데이트 **(**페이스 서버**2)**
두 번째 핵심 코드는 지정된 폴더**(/home/moble03/face)**에 저장된 사용자들의 사진을 읽어와 얼굴 특징**(**임베딩**)**을 미리 추출하고 평균값을 계산하여 메모리에 저장하는 기능입니다**.** 이 과정을 통해 실시간으로 들어오는 영상과 빠르고 정확하게 비교할 수 있습니다**.**

코드 설명

서버가 시작되거나 재학습 요청이 있을 때**,** 각 사용자 폴더에 있는 모든 사진에서 얼굴을 찾습니다**.**

alignCrop으로 얼굴을 정렬하고 **feature**로 고유한 특징 벡터**(**임베딩**)**를 추출합니다**.**

한 사용자의 여러 사진에서 추출된 특징 벡터들의 평균값을 계산하여**,** 조명이나 각도 변화에 더 강인한 대표 특징을 만듭니다**.**

이 대표 특징들을 **registered_user_embeddings** 맵에 저장해두고**, /detect API** 요청 시 비교 데이터로 사용합니다**.**

현수 파트 코드 부분

```
// --- EAR (눈 종횡비) 계산 함수 ---
double calculate_ear(const std::vector<dlib::point>& eye_points) { [cite: 320]
    // ... 눈의 세로, 가로 길이 비율 계산 ...
    return (v1 + v2) / (2.0 * h); [cite: 322]
}

// --- MAR (입 종횡비) 계산 함수 ---
double calculate_mar(const std::vector<dlib::point>& mouth_points) { [cite: 323]
    // ... 입의 세로, 가로 길이 비율 계산 ...
    return (v1 + v2 + v3) / (3.0 * h); [cite: 325]
}

// detect API 내부 로직 중 일부...
auto& state = user_states[target_name];
if (avg_ear < EAR_THRESHOLD) state.drowsy_frame_counter++; else state.drowsy_frame_counter = 0; [cite: 397]
if (mar > MAR_THRESHOLD) state.yawn_frame_counter++; else state.yawn_frame_counter = 0; [cite: 397-398]

bool drowsy_alert = state.drowsy_frame_counter >= DROWSY_CONSEC_FRAMES; [cite: 399]
bool yawn_alert = state.yawn_frame_counter >= YAWN_CONSEC_FRAMES; [cite: 399]
if (drowsy_alert) {
    cv::putText(frame, "!!! DROWSY !!!", ...); [cite: 399]
    is_drowsy_now = true; [cite: 400]
}
```

3. 실시간 졸음 감지 및 사용자 추적 **(**레지스터 페이스**)**
세 번째 핵심 코드는 실시간 영상에서 단순히 얼굴을 인식하는 것을 넘어, **dlib** 라이브러리를 사용해 눈과 입의 랜드마크를 정밀하게 추적하고, 이를 기반으로 졸음 상태**(**눈 깜빡임, 하품**)**를 판단하는 기능입니다. 또한, 여러 프레임에 걸쳐 사용자를 안정적으로 추적하는 로직도 포함됩니다.

코드 설명

calculate_ear (Eye Aspect Ratio) 함수는 눈의 **6**개 랜드마크를 이용해 눈을 얼마나 뜨고 있는지를 수치화합니다. 이 값이 특정 임계값**(EAR_THRESHOLD)**보다 일정 시간**(DROWSY_CONSEC_FRAMES)** 동안 낮게 유지되면 '졸음'으로 판단합니다.

calculate_mar (Mouth Aspect Ratio) 함수는 입의 **8**개 랜드마크로 입을 벌린 정도를 계산하여 '하품' 여부를 감지합니다.

이 기능은 원격 근무자의 업무 집중도를 실시간으로 모니터링하는 핵심적인 역할을 하며, 단순한 본인 인증을 넘어 근무 상태를 관리하는 고도화된 기능을 제공합니다.

현수 파트 코드 부분



예시

www.BANDICAM.com

(주)더문

업무 관리 시스템

사용자 로그인

시스템을 이용하시려면 로그인해주세요.

아이디

비밀번호

로그인

◉ 일반 계정 ○ 관리자

계정이 없으신가요? 회원가입

© 2025 The Moon Inc. All Rights Reserved.

자체 평가 의견

완성도 평가

협업 효율 극대화를 위한 "통합 업무 플랫폼 구축"이 원활하게 이루어졌는가? 9/10

투명한 근무 관리를 위한 "지능형 모니터링 시스템구축"이 원할하게 이루어졌는가? 9/10

잘한 점

무거운 작업은 성능이 빠른 C++ 서버에 맡기고, 웹 화면 표시와 사용자 관리는 Python Flask 서버가 담당하도록 역할을 명확히 나눴음.

직관적으로 MFC UI를 만들었고, 부드럽게 만들었음. 유지보수가 편리하게 만들었음.

보완할 점

통신 방식은 HTTPS에서 웹소켓으로 변경할 예정. MFC가 DB에 직접 연결하는 대신, Flask 서버를 통해 연결할 것임.

성능 문제는 라즈베리파이의 OpenCV 얼굴 인식 기능 저하 및 잔랙 때문.
해결책은 고성능 리눅스 컴퓨터로 교체해야 함

팀 내 성과

다양한 기술 스택을 사용하여 코딩에 대해 점층적으로 이해하게 되었음.

역할 분배의 중요성을 크게 느끼고, 앞으로의 프로그램 개발에 도움이 될것임

소감

강승진

첫 팀장으로서 많은 힘듦과 고단함을 겪었으나, 팀원들과 함께 협력하여 모든 난관을 극복하고 프로젝트를 완수했습니다.
특히, 이 경험을 통해 처음 접하는 다양한 개발 툴들의 활용법을 깊이 있게 숙달할 수 있었습니다. 이 귀중한 경험과 기술적 성취는 앞으로 제가 마주할 모든 개발 활동에 긍정적인 밑거름이 될 것이라 확신합니다.

김주엽

프로젝트의 중심인 플라스크 서버를 직접 다루며 전체적인 데이터 흐름을 주도적으로 파악할 수 있었습니다. 웹 요청을 받아 라즈베리파이의 LED를 제어하는 과정에서 통신 지연 문제가 발생했지만, API 구조를 개선하며 문제를 해결하고 웹과 하드웨어를 잇는 값진 경험을 했습니다.

김찬송

공지사항, 커뮤니티, 메신저 기능을 갖춘 MFC 플랫폼을 완성하며 개발 역량을 한 단계 성장시켰습니다. 이 모든 과정은 서로 격려하고 이끌어준 우리 팀원들이 없었다면 불가능했을 것이기에 깊은 감사를 전합니다. 함께였기에 성공할 수 있었고, 협력을 통해 더 큰 시너지를 만드는 값진 경험을 했습니다

송현수

회의 후 바로 긴 연휴가 이어져 작업 흐름이 끊기는 어려움이 있었으나, 복귀 후 집중적인 개발로 시스템을 성공적으로 완료했습니다.
다음 프로젝트에서는 휴식 기간 전후의 일정 관리를 더욱 철저히 해야 할 것 같습니다. 좋은 팀원들을 만나고 부족한 저를 이끌어주었기때문에 뜻이 깊은 프로젝트였습니다!